Machine Learning in Computational Biology

Assignment 1 - Epigenetic Age Prediction from DNA Methylation

Panagiotis Giannis, 7115152500023 - March 2026

1. Introduction

The purpose of this project is the development of a machine learning pipeline for epigenetic age prediction from DNA methylation data by utilizing the publicly available GSE40279 dataset (Hannum et al., 2013), measuring whole-blood DNA methylation using the Illumina 450K microarray. The original dataset comprises 656 samples from individuals aged 19–101 years. The 450K array measures beta values, the proportion of methylated reads at each CpG site, ranging from 0 (fully unmethylated) to 1 (fully methylated), at approximately 480,000 CpG positions. For this assignment, the dataset has been pre-processed into the files development_data.csv and evaluation_data.csv available on GitHUb. The pipeline begins with basic data loading, splitting and preprocessing, continues with the training of three OLS baselines and the three linear models ElasticNet, SVR and BayesianRidge, before attempting to maximize the latter ones' performance via feature selection and hyperparameter tuning.

2. Materials and Methods

2.1- Data loading & splitting

To begin the pipeline, the files development_data.csv and evaluation_data.csv that constitute the dataset for this assignment were downloaded locally and then loaded into the python environment as data frames. Afterwards, the development data frame was split into two subsets, the training set and the validation set, each containing 80% and 20% of the development data respectively. To ensure balanced sets and therefore comparable results, the continuous age values (19 - 101 years) were divided into 10 quantile bins each containing 10% of the samples which were used to stratify the dataset before splitting it, effectively avoiding age bias.

2.2- Exploratory analysis

To gain a better understanding of the dataset, a function was built that prints the number of samples and features, along with the age mean and range and the sex counts of the data frame. After calling the function for the three datasets (training, validation and evaluation), the results are shown in the table below, followed by the age distribution of the development dataset.

Statistics	**Training**	**Validation**	**Evaluation**
Number of Samples	364	92	100
Number of Features	1003	1003	1003
Age Mean ± STD	64.5 ± 14.6	64.3 ± 14.8	62.5 ± 14.3
Age Range	19 - 101	22 - 92	21 - 92
Number of Females	187	47	58
Number of Males	177	45	42

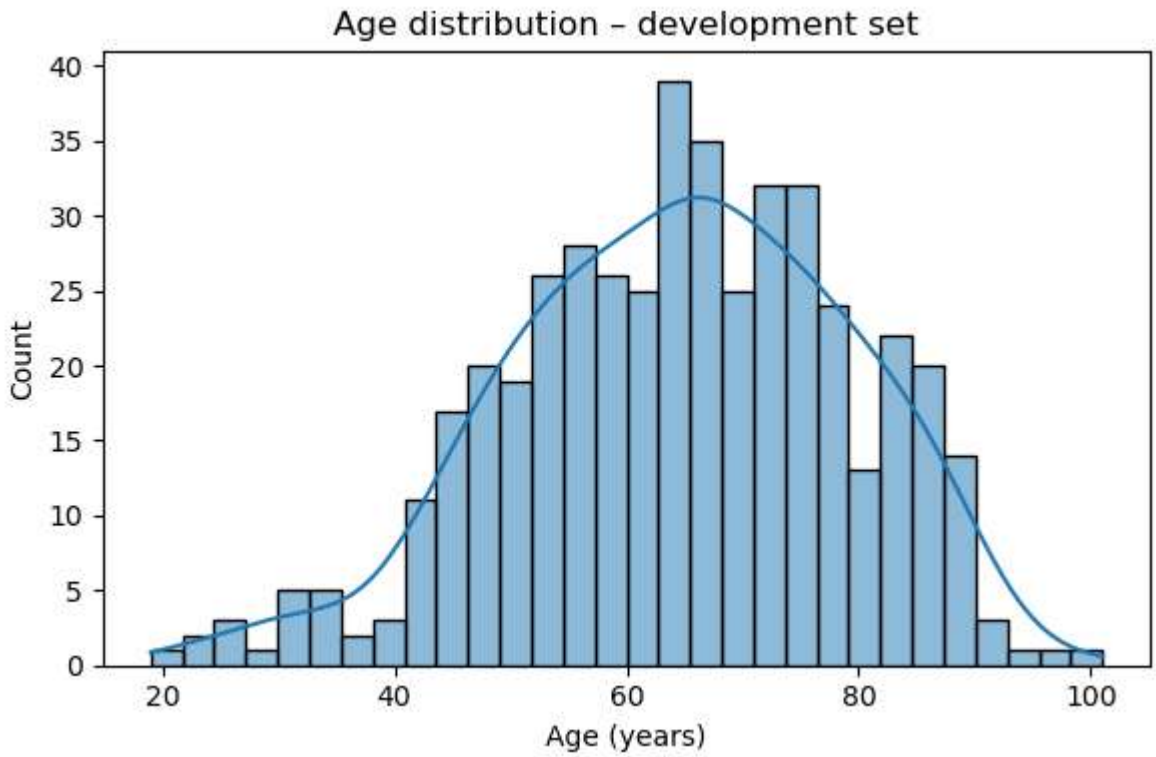

Image 2.1*: Age distribution of the development set, showing a slightly right-skewed normal distribution, indicating a well-balanced dataset.*

2.3- Feature Matrices

Before any preprocessing can take place the feature matrices on which the pipeline will be applied must first be constructed. For that reason, the function get_X_y() was created that takes

three arguments, one for which data frame to use and two Boolean ones that inform it whether to use the CpG data or the metadata. Specifically, it sets the "age" column as the target, the "sex" and "ethnicity" columns as the categorical columns (metadata) and detects the CpG features by their common "cg" prefix. After that, it was deemed necessary to be able to select which features to include downstream in the pipeline by introducing two 'if" statements that include, either the cpg data, the metadata or both, depending on the boolean values of the function arguments "use_cpg" and "use_metadata".

2.4- Preprocessing

Moving on, the dev_df was inspected for values that are Missing Completely at Random (MCAR) and it was found that only the CpG features had missing values, unlike the metadata columns ("sex" and "ethnicity") and the age column. Despite that, it was decided to impute all features, regardless, to futureproof the pipeline in the case it is ever applied on a dataset that has missing values of the categorical features as well. For that reason, the build_preprocessing_pipeline() function was built that, after categorizing the column groups and selecting which features to include, as in the previous function, it creates a scikit-learn pipeline.

Specifically, given the random nature of the missing values, it was decided that the **mean** is the most appropriate strategy of imputation for the numerical features, since the missingness is independent of the feature's value, other features and the target variable. Along with that, StandardScaler was applied to the same CpG features since it is beneficial for the numerical stability of linear models. Regarding the "sex" and "ethnicity" categorical features the "most_frequent" strategy was chosen that imputes missing values with the most common category and were then one-hot encoded. Finally, the results of the above preprocessing were concatenated into a single feature matrix and the pipeline was created.

2.5- Baselines & Regression Models

To begin the training of the OLS models, the three feature sets were created by applying the get_X_y function, defined in 2.3, one containing only CpG data, another with only metadata and another with both. Then the build_preprocessing_pipeline() function, built in 2.4 was applied along with the helper function "build_model_pipeline", which takes as arguments the preprocessing pipeline and the model and combines them into a unified pipeline to avoid data leakage. Afterwards, a bootstrap evaluation function was created that collects 1000 bootstrap samples and saves the results in data frames to build the required RMSE and R^2 boxplots. On top of that, another function was introduced, that calculates and prints the mean values of RMSE, MAE, R^2, and Pearson r with 95% confidence intervals (CI) for every model. Having trained the

baseline, the regression models ElasticNet, SVR and BayesianRidge were also trained at default parameters and only on the CpG feature set.

2.6- Stability Selection

The first feature selection method used in this pipeline is stability selection, which was chosen to estimate the robustness of feature rankings across data perturbations. Specifically, 50 subsamples of size equal to 80% of the training data, without replacement were drawn and for each one all CpG features were ranked by Spearman correlation, with age. Out of these, the top 200 were chosen and were then filtered to keep only the stable ones, meaning those that appeared in more than half of the 50 resamples. Finally, a function was built to evaluate the stability-selected features, by using a BayesianRidge as a proxy model on the validation set and produce the relevant statistics.

2.7- Minimum Redundancy Maximum Relevance (mRMR)

Due to the previous method's incapability of handling inter-feature redundancy, the mRMR method was applied for several K values (50, 100, 200 and 300), to address this problem, by optimizing maximum relevance and minimum redundancy. This way, each selected feature should have high correlation with the target variable (age) and selected features should be minimally correlated with each other. Having done that, a function was created that returns the top 10 features with the highest mMRM scores. To finish the mRMR methodology, the same evaluation function as with stability selection was applied for the different feature sets and the produced statistics were used to compare them.

2.8- Hyperparameter Tuning

The feature set found previously was used to tune the hyperparameters of all three algorithms using RandomizedSearchCV with 5-fold cross-validation on the development set only, to minimize cross-validation RMSE. Therefore, after building the development and evaluation feature matrices, applying the preprocessing pipeline and defining the model pipelines, the minimum search spaces were also defined as instructed. Afterwards, a function was built that uses RandomizedSearchCV, with 40 iterations and 5-fold CV, which was applied for each model pipeline and search space and the best models were refitted on the full development set.

Model	**Key hyperparameters to search**
ElasticNet	alpha (log-uniform, 0.001–10) · l1_ratio (uniform, 0.1–1.0)

SVR	C (log-uniform, 0.1–500) · epsilon {0.01, 0.1, 0.5, 1.0} · kernel {rbf, linear}
BayesianRidge	alpha_1, alpha_2, lambda_1, lambda_2 (log-uniform, 1e-7–1e-3)

3. Results

3.1 - Model Training Comparison

Below are the RMSE and R^2 boxplots, for each model (the three OLS baselines and the three regression models), followed by a table containing the average statistics; all values have been rounded up to the second decimal point. According to these results all regression models beat the floor baseline, metadata-only OLS, which is to be expected since it does not utilize the CpG data and is limited to predicting the age based on relatively uninformative features such as the ethnicity and the sex. As far as the other two baselines are concerned, the CpG-only OLS performs slightly worse than the one that utilizes metadata as well, which indicates that the either one or both of the categorical features included in the metadata improve the predictive capabilities of a model when included along with the main CpG data.

Regarding the performance of the regression models, the BayesianRidge outperforms every other model, including all baselines in all metrics, making it by far the best age predictor when trained on the CpG-only feature set. Second best is the ElasticNet, which also outperforms all OLS baselines in practically all aspects, unlike the SVR which is the worst model out of the ones trained on the CpG-only feature set and is a worst age predictor than even the CpG-only and CpG + metadata baselines.

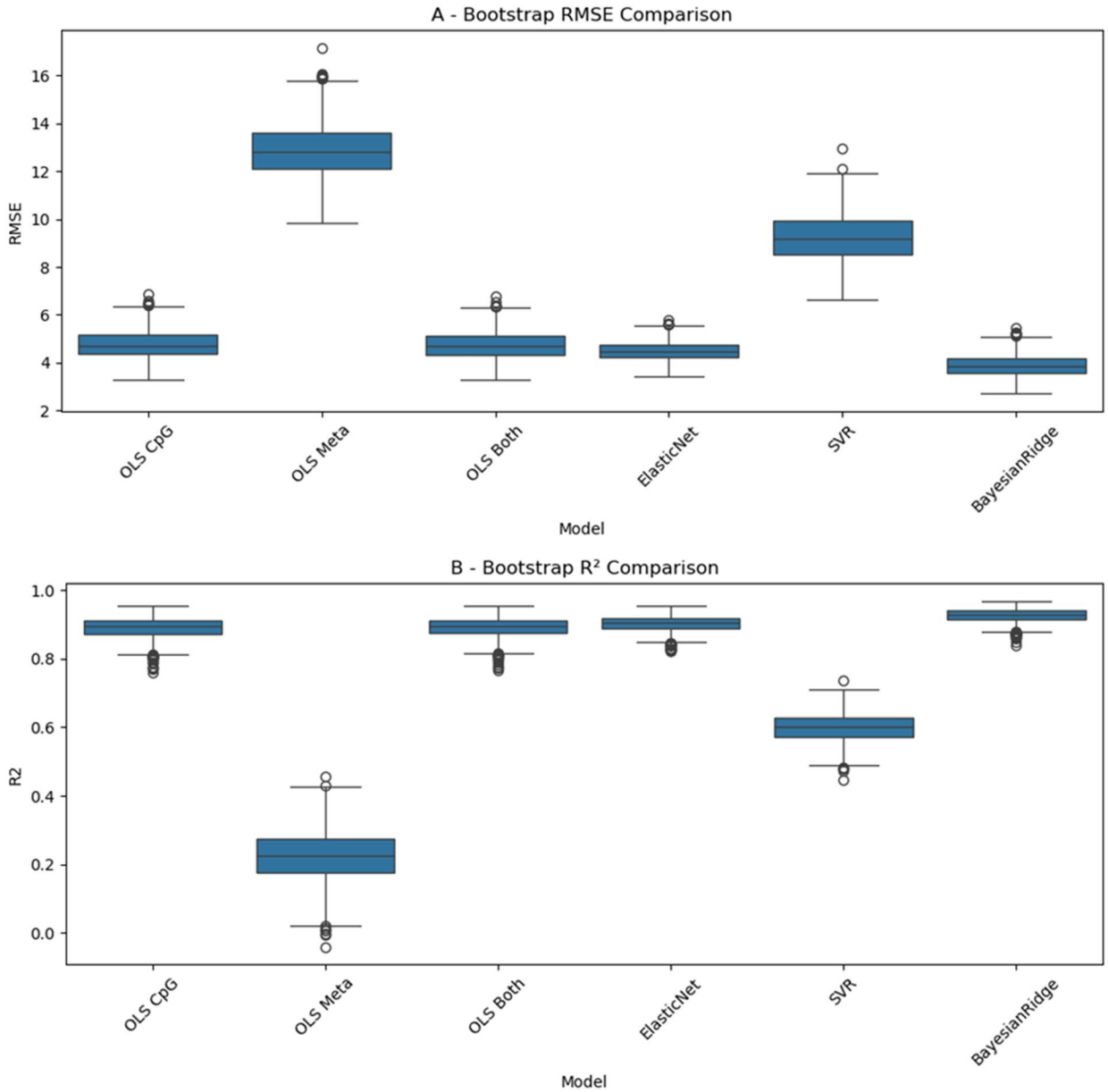

Image 3.1*: RMSE (A) and R^2 (B) bootstrap boxplots comparing the three OLS baseline models and the three regression models. The metadata-only OLS (OLS Meta) is the floor baseline and the OLS CpG + metadata (OLS both) outperforms the OLS CpG-only (OLS CpG). The BayesianRidge is the best model, followed by the ElasticNet and the SVR, which collapses at default values and only beats the floor baseline.*

Model	**Stage**	**Mean RMSE**	**95% CI**	**Mean MAE**	**Mean R^2**	**Mean Pearson r**
OLS	CpG-only	4.77	[3.7, 5.98]	3.58	0.89	0.95
OLS	Metadata-only	12.85	[10.8, 15.3]	10.35	0.22	0.48
OLS	CpG + metadata	4.73	[3.69, 5.91]	3.55	0.89	0.95
ElasticNet	CpG-only	4.49	[3.77, 5.3]	3.47	0.90	0.95

SVR	CpG-only	9.25	[7.4, 11.22]	6.79	0.6	0.85
BayesianRidge	CpG-only	3.89	[3.11, 4.8]	2.91	0.93	0.96

3.2- Stability Selection Results

The total number of stable features amounted to 200 and their selection-frequency distribution is shown in the plot below, where two spikes are most notable. One corresponding to around 800 CpGs, near frequency = 0, indicating weak or inconsistent association with age (noise) and a smaller one, around 200 CpGs, near frequency = 50, indicating the stable features that were ultimately kept.

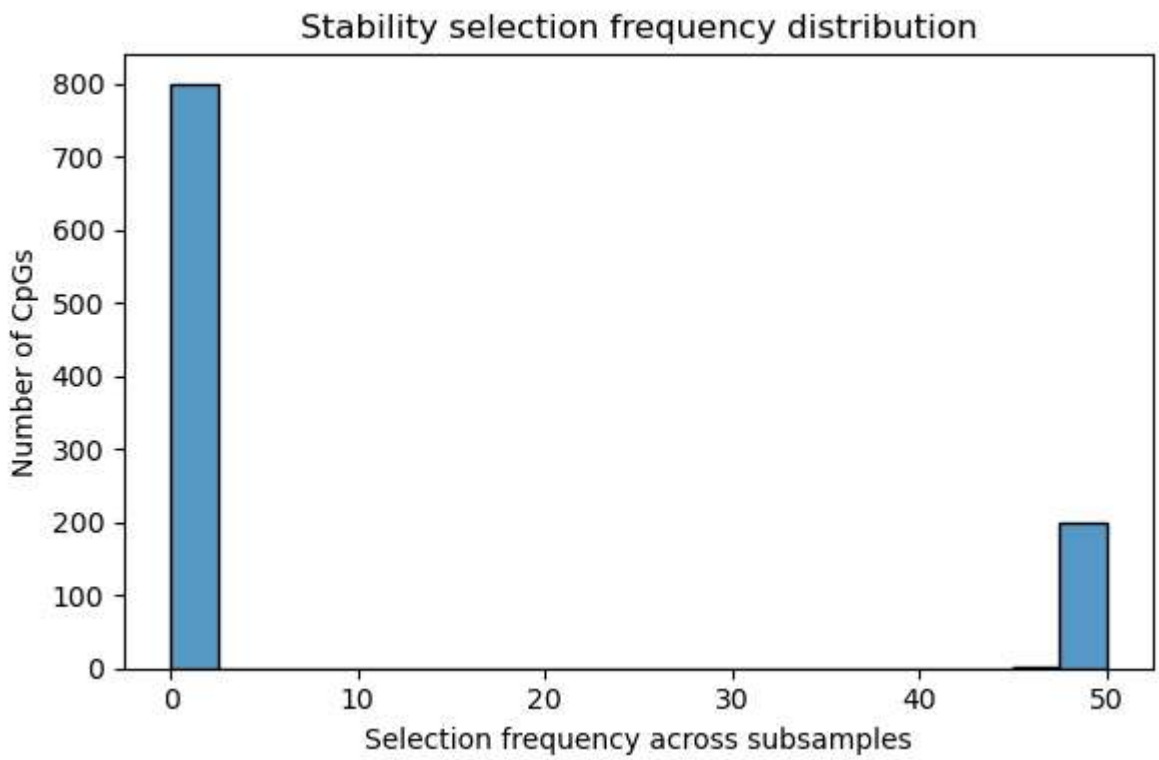

Image 3.2*: Selection frequency distribution of stable features; two spikes are noticeable, one near frequency = 0, corresponding to the noise and another near frequency = 50, corresponding to the selected stable features.*

3.3- mRMR Results

Below are shown the top 10 features ranked by mRMR (importance) score and a table containing the mean RMSE and R2 values for each of the 4 feature sets created by the different K values. The feature set created with K = 300 has the lowest RMSE value and is therefore considered to be the best one.

Feature	Importance Rank
cg16867657	1
cg22454769	2
cg10501210	3
cg06639320	4
cg24724428	5
cg21572722	6
cg22796704	7
cg24079702	8
cg01974375	9
cg08097417	10

K Value	Mean RMSE	Mean R^2
50	5.43	0.86
100	4.8	0.89
200	4.22	0.91
300	4	0.92

3.4- Feature Selection Methods Comparison

By combining the previous results, the following table is produced which contains the number of features, the mean RMSE and R^2 for the two methods, along with the K of the best feature set of mRMR. It is clear that mRMR outperforms stability selection, since it has both a lower RMSE value and a higher R^2 value, even though mRMR uses more features. The reason behind this difference in predictive accuracy lies in the pre-mentioned advantage of mRMR, regarding the handling of inter-feature redundancy, that stability selection ignores. This is better shown by the few (29), redundant stability-only features shown in the following Venn diagram (red color). Despite the difference in performance, the two methods share 171 features, as shown by the large overlap in the same diagram (brown color), which represents the most biologically and statistically robust CpGs, that are highly correlated with age and consistent across subsamples.

Method	Number of Features	K Value (mRMR)	Mean Validation RMSE	Mean Validation R^2
Stability Selection	200	-	4.35	0.9
mRMR	300	300	4	0.92

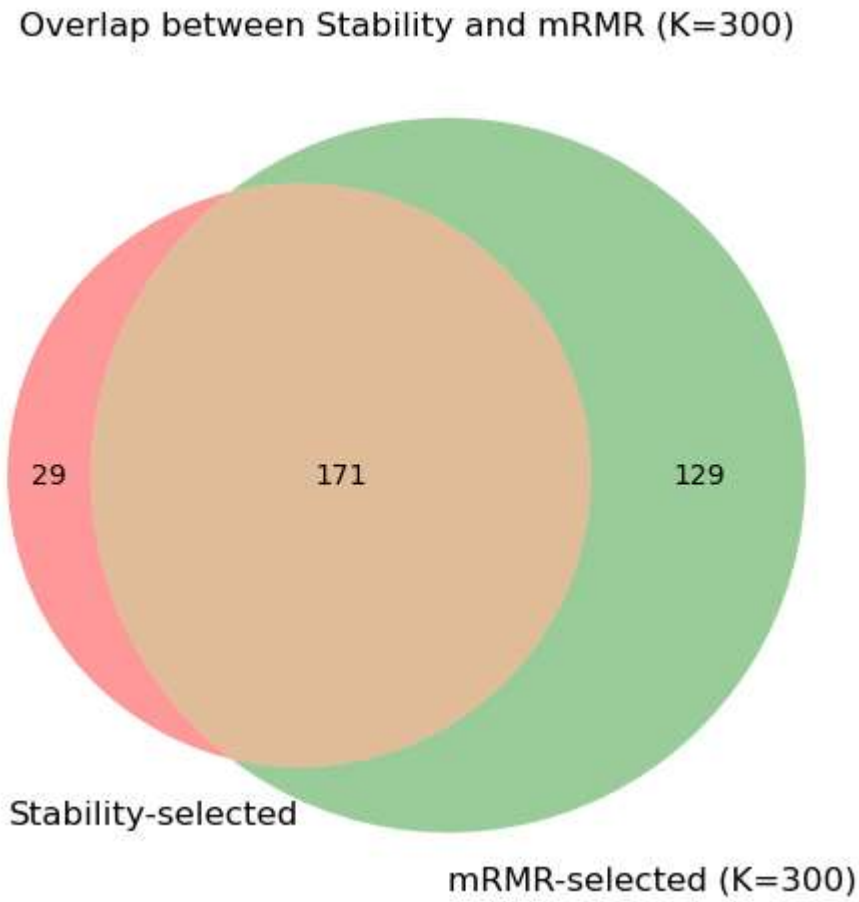

Image 3.3*: Venn diagram of feature overlap between stability selection and mRMR (K = 300); with red are shown the stability-only, redundant features that drop the performance of the method, with green the mRMR-only features and with brown the shared features.*

3.5- Model Comparison on the Feature Set

Having found the best feature set, it was deemed useful to train the three linear models on it to compare with the previous comparison table, that utilized the entire training set. It is evident that, practically only the SVR benefited from the feature selection since it's RMSE dropped from 9.25 to 8.65, unlike the ElasticNet which remained stable and the BayesianBridge which surprisingly underperformed slightly (3.89<3.95). That being said, it is possible that the ElasticNet and the BayesianRidge have already reached their best predictive capabilities and feature selection or even hyperparameter tuning may not make them any better.

Model	**Stage**	**Mean RMSE**	**95% CI**	**Mean MAE**	**Mean R^2**	**Mean Pearson r**
ElasticNet	Feature Set	4.5	[3.77, 5.28]	3.48	0.9	0.95
SVR	Feature Set	8.65	[6.8, 10.77]	6.21	0.65	0.86

BayesianRidge	Feature Set	3.95	[3.34, 4.55]	3.01	0.92	0.96

3.6- Model Evaluation Comparison

Below is a table summarizing the statistics for the three linear models after being trained on the feature set, with tuned hyperparameters and evaluated on the evaluation set. It is evident that, tuning only improves the performance of the SVR, which is still the worst among the tuned models. The ElasticNet, slightly outperforms the BayesianRidge, in terms of mean RMSE and mean MAE. For the better comparison of the tuned models two bootstrap boxplots, one for RMSE and one for R^2, as well as a scatterplot of predicted vs actual age have been created for each model, where it is again shown that the ElasticNet and the BayesianRidge are practically equally the best performers. It should be noted that the the BayesianRidge experiences a notable drop in performance when applied on the evaluation set (RMSE decreases from 3.95 vs 4.84), but it is expected, and is not associated with overfitting, because the model generalizes regardless and the mean R^2 remains practically stable (0.92 vs 0.88).

Model	**Stage**	**Mean RMSE**	**95% CI**	**Mean MAE**	**Mean R^2**	**Mean Pearson r**
ElasticNet	Feature Set + Tuned	4.8	[3.6, 6.5]	3.38	0.88	0.94
SVR	Feature Set + Tuned	5.26	[4.17, 6.67]	3.94	0.86	0.93
BayesianRidge	Feature Set + Tuned	4.84	[3.61, 6.45]	3.41	0.88	0.94

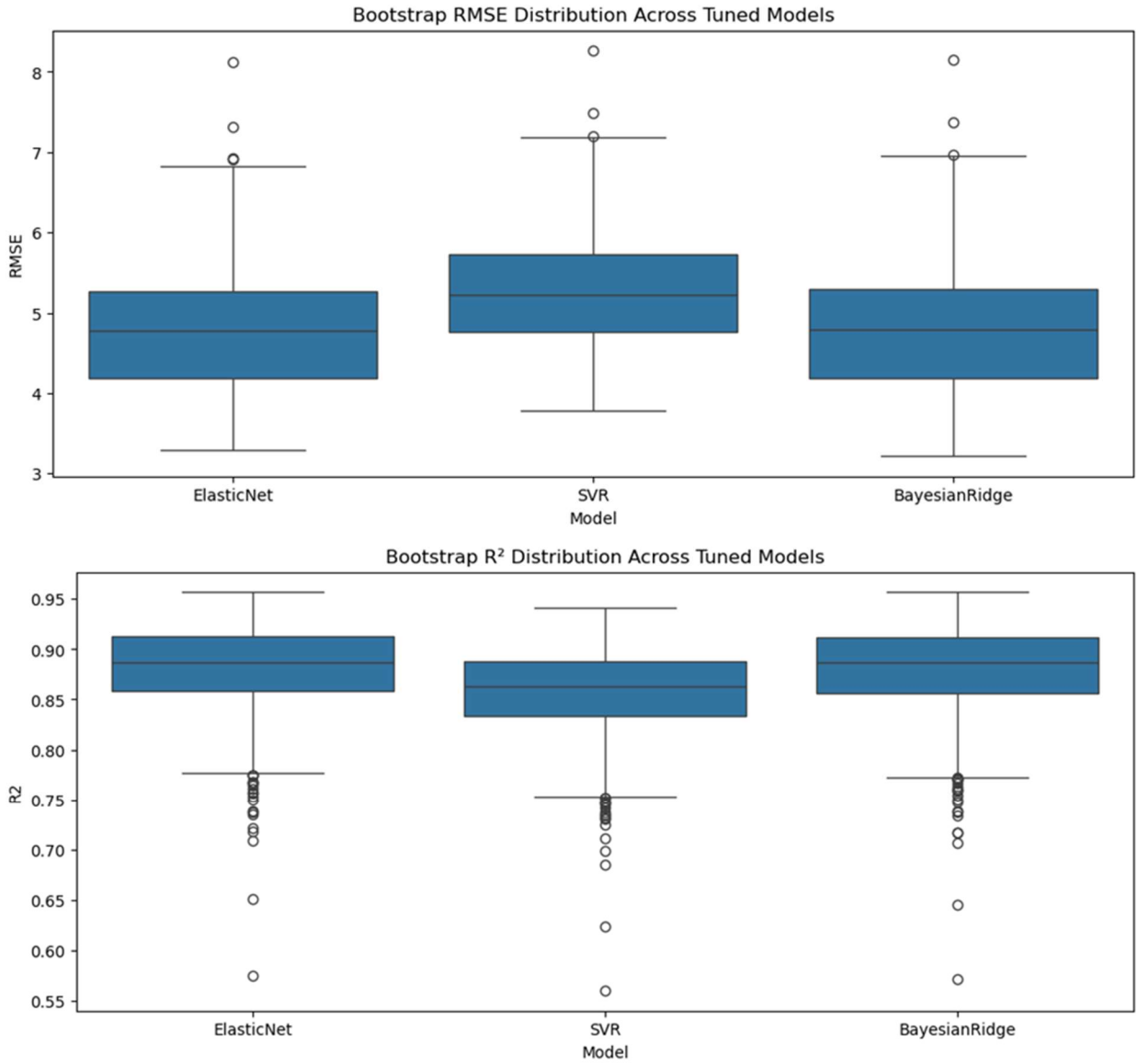

Image 3.4*: RMSE (A) and R^2 (B) bootstrap boxplots comparing the three regression models when trained on the best feature set and after hyperparameter tuning. The BayesianRidge and the ElasticNet are practically the best performing models and practically equal in their metrics.*

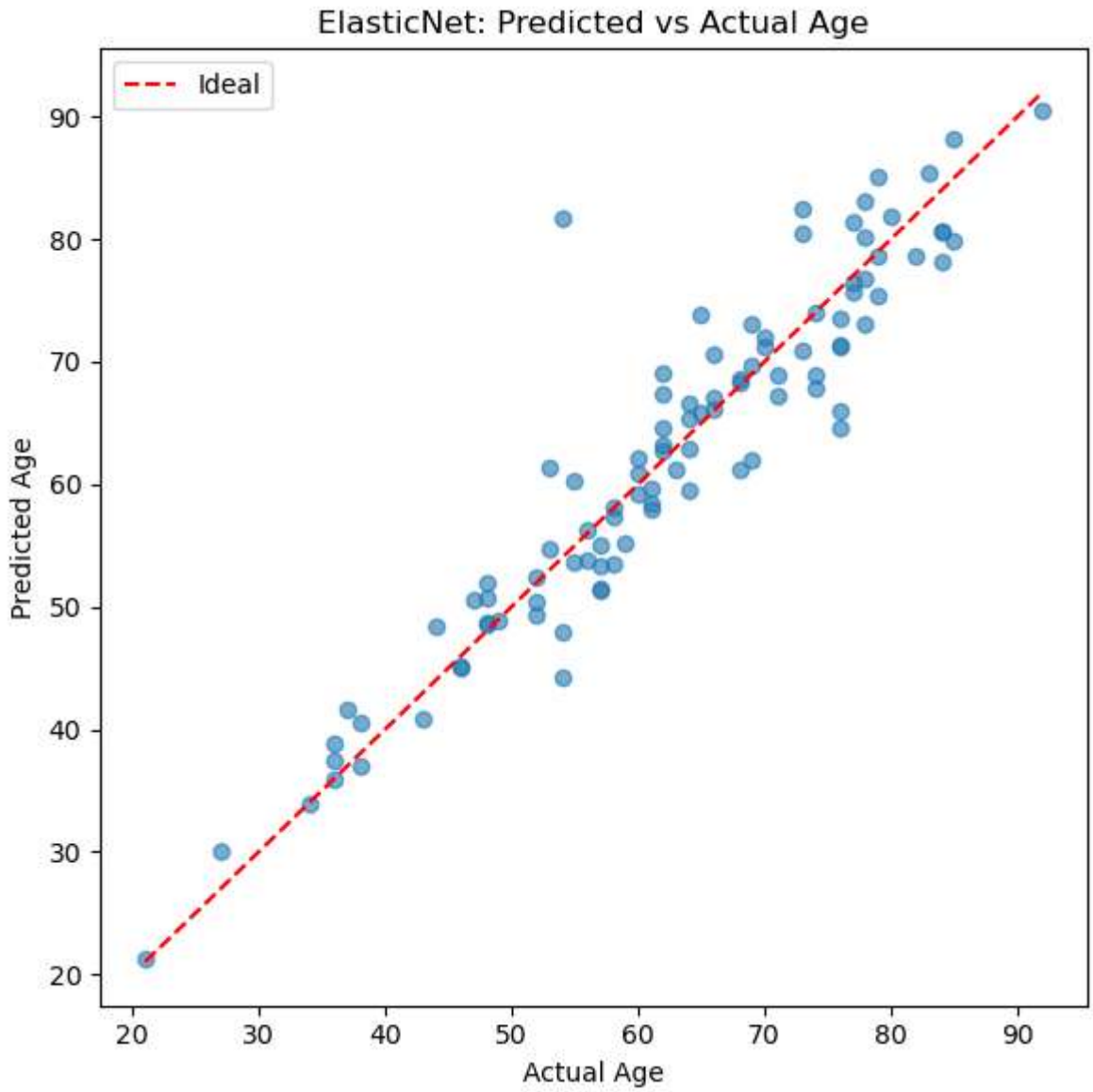

Image 3.5*: Scatterplot of predicted vs actual age for the ElasticNet after hyperparameter tuning.*

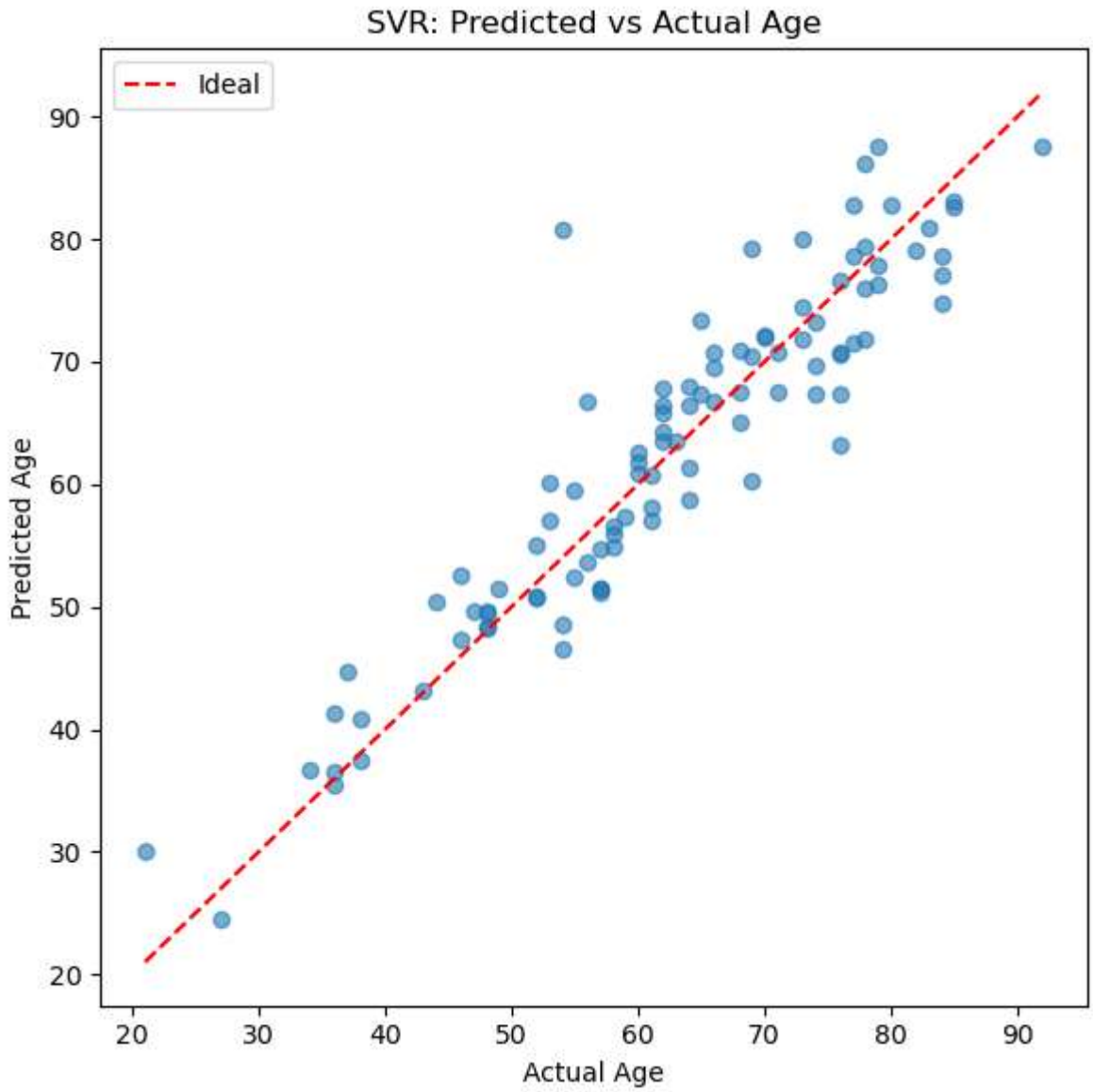

Image 3.6*: Scatterplot of predicted vs actual age for the SVR after hyperparameter tuning.*

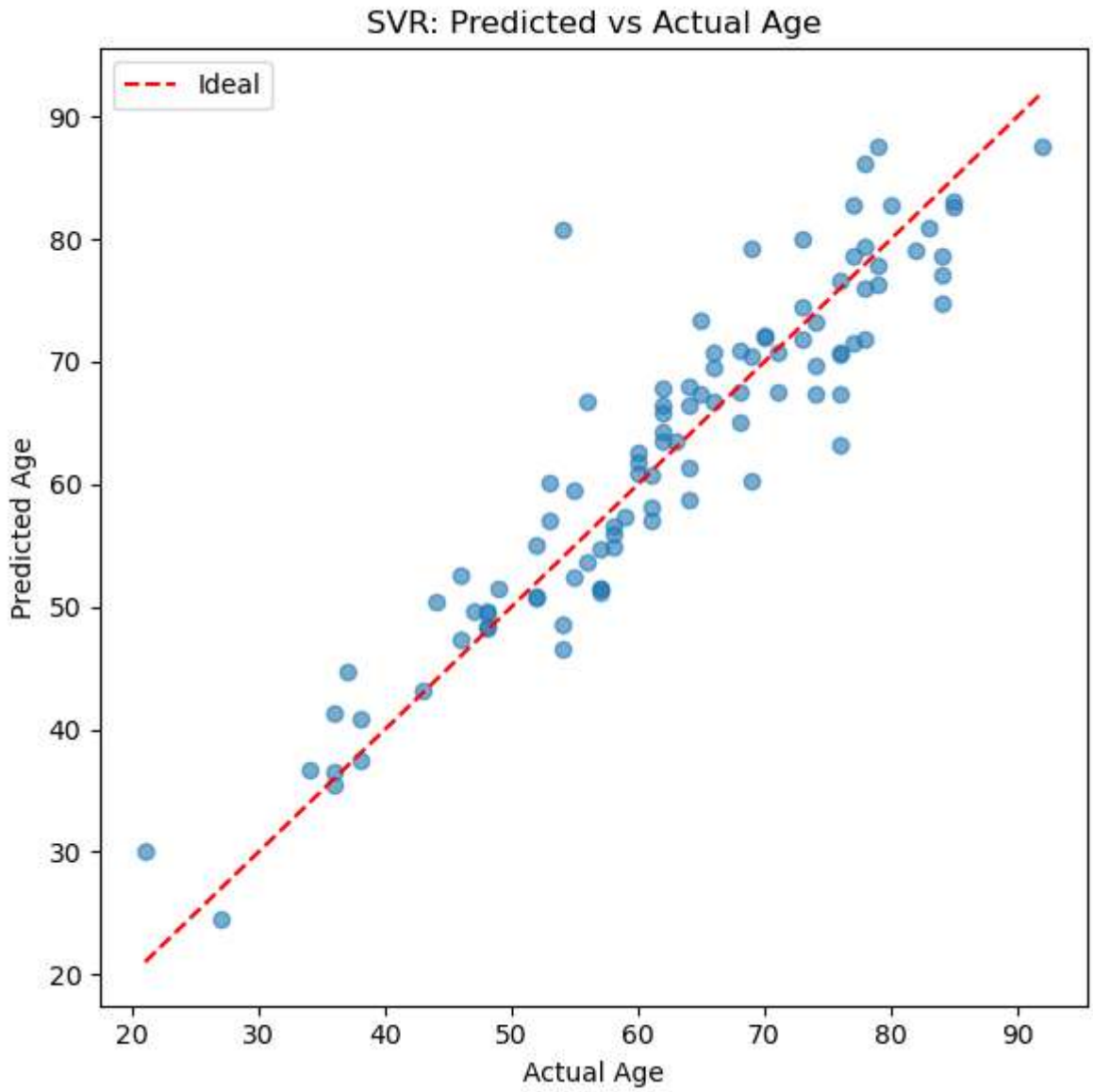

Image 3.7*: Scatterplot of predicted vs actual age for the BayesianRidge after hyperparameter tuning.*

4. Conclusions

Having completed the entire machine learning pipeline for this project and studying the results, two notable conclusions can be drawn about the performance of the used models and the effects of feature selection and hyperparameter tuning. Firstly, among the three linear models, the SVR is consistently the worst age predictor, surpassing only the floor baseline, but it is also the model with the best progress, since practically all its metrics improve after feature selection and hyperparameter tuning. Among the two remaining contenders, it has been decided that the

BayesianRidge is the best model for this task, due to its consistently superior performance from the start, unlike the ElasticNet that only matches it at the end, but even if that higher stability is ignored, the BayesianRidge is still better, because it achieves practically the same results on the evaluation set, but with lower model complexity.

5. Appendix

The code used on this project, along with the data, the best models and the figures can be found in this GitHub repository, https://github.com/Pangi898/Machine-Learning-in-Computational-Biology---Assignment-1.git. It should be noted that the LLM Copilot has been used in the writing of the code only and not in the writing or interpretation of results.